padufes20_split

August 10, 2026

[1]:
```
import os
import pandas as pd
import numpy as np
```

[2]:
```
TRAIN_RATIO = 0.7
VAL_RATIO = 0.1
TEST_RATIO = 0.2
RANDOM_SEED = 123
```

[3]:
```
rng = np.random.default_rng(RANDOM_SEED)
```

[4]:
```
assert abs(TRAIN_RATIO + VAL_RATIO + TEST_RATIO - 1.0) < 1e-6
```

[5]:
```
df = pd.read_csv("padufes20_metadata_official.csv")
```

[6]:
```
df.columns
```

[6]:
```
Index(['patient_id', 'lesion_id', 'smoke', 'drink', 'background_father',
       'background_mother', 'age', 'pesticide', 'gender',
       'skin_cancer_history', 'cancer_history', 'has_piped_water',
       'has_sewage_system', 'fitspatrick', 'region', 'diameter_1',
       'diameter_2', 'diagnostic', 'itch', 'grew', 'hurt', 'changed', 'bleed',
       'elevation', 'img_id', 'biopsed'],
      dtype='str')
```

[7]:
```
keep_cols = [
    "img_id",
    "diagnostic",
    "patient_id",
    "age",
    "gender",
    "region",
]
df = df[keep_cols]
```

[8]:
```
df["diagnostic"].value_counts(dropna="FALSE")
```

[8]:
```
diagnostic
BCC    845
ACK    730
NEV    244
SEK    235
SCC    192
MEL     52
Name: count, dtype: int64
```

[9]:
```
df["dataset"] = "padufes20"
```

[10]:
```
df.rename(columns={
    "img_id": "image_name",
    "diagnostic": "label",
    "gender": "sex",
    "region": "location",
}, inplace=True)
```

[11]:
```
df["sex"] = df["sex"].fillna("unknown")
```

[12]:
```
df.isna().sum()
```

[12]:
```
image_name    0
label         0
patient_id    0
age           0
sex           0
location      0
dataset       0
dtype: int64
```

[13]:
```
assert df.isnull().sum().sum() == 0
```

[14]:
```
assert len(df) == df["image_name"].nunique()
```

[15]:
```
missing = []
for f in df["image_name"]:
    if not os.path.exists(os.path.join("/home/prasad/datasets/padufes20/images/
 ↪", f)):
        missing.append(f)

print(len(missing))
```

```
0
```

[16]:
```
sorted(df["label"].unique().tolist())
```

[16]:
```
['ACK', 'BCC', 'MEL', 'NEV', 'SCC', 'SEK']
```

[17]:
```
mapping = {
    "ACK": "AKIEC",
    "BCC": "BCC",
    "MEL": "MEL",
    "NEV": "NEV",
    "SCC": "SCC",
    "SEK": "SEK",
}
```

[18]:
```
assert set(df["label"].unique()) == set(mapping.keys())
```

[19]:
```
df["label"] = df["label"].map(mapping)
```

[20]:
```
assert df["label"].isna().sum() == 0
```

[21]:
```
assert sorted(df["label"].unique()) == [
    "AKIEC", "BCC", "MEL", "NEV", "SCC", "SEK"]
```

[22]:
```
df["label"].value_counts()
```

[22]:
```
label
BCC      845
AKIEC    730
NEV      244
SEK      235
SCC      192
MEL       52
Name: count, dtype: int64
```

[23]:
```
print(df["label"].value_counts().sum())
```

```
2298
```

[24]:
```
df["binary_label"]=(df["label"].isin(
    ["AKIEC", "MEL", "BCC", "SCC"])).astype(int)
```

[25]:
```
assert set(df["binary_label"].unique()) == {0,1}
```

[26]:
```
print(df["binary_label"].value_counts())
```

```
binary_label
1    1819
0     479
Name: count, dtype: int64
```

[27]:
```
print(df["binary_label"].value_counts().sum())
```

```
2298
```

[28]:
```
print(pd.crosstab(df["binary_label"], df["label"]))
```

```
label         AKIEC  BCC  MEL  NEV  SCC  SEK
binary_label
0                 0    0    0  244    0  235
1               730  845   52    0  192    0
```

[29]:
```
df["sex"].value_counts()
```

[29]:
```
sex
unknown    804
FEMALE     753
MALE       741
Name: count, dtype: int64
```

[30]:
```
print(df["sex"].value_counts().sum())
```

```
2298
```

[31]:
```
print(pd.crosstab(df["sex"], df["label"]))
```

```
label    AKIEC  BCC  MEL  NEV  SCC  SEK
sex
FEMALE     153  406   28   49   90   27
MALE       130  439   24   26  102   20
unknown    447    0    0  169    0  188
```

[32]:
```
df["age"].value_counts()
```

[32]:
```
age
55    73
73    73
58    70
75    66
62    66
      ..
8      2
24     2
12     2
6      2
18     1
Name: count, Length: 84, dtype: int64
```

[33]:
```
df["age"].value_counts().sum()
```

[33]:
```
np.int64(2298)
```

[34]:
```
df["age"].isna().sum()
```

[34]:
```
np.int64(0)
```

[35]:
```
print(pd.crosstab(df["age"], df["label"]))
```

```
label  AKIEC  BCC  MEL  NEV  SCC  SEK
age
6          0    0    0    2    0    0
8          0    0    0    2    0    0
9          0    0    0    3    0    0
10         0    0    0    5    0    0
12         0    0    0    2    0    0
..       …  …  …  …  …  …
89         2    4    0    0    0    1
90         2    3    2    0    4    3
91         0    5    0    0    2    0
92         1    0    0    0    3    0
94         0    2    0    0    1    0

[84 rows x 6 columns]
```

[36]:
```
df["location"].value_counts()
```

[36]:
```
location
FACE       570
FOREARM    392
CHEST      280
BACK       248
ARM        192
NOSE       158
HAND       126
NECK        93
THIGH       73
EAR         73
ABDOMEN     36
LIP         23
SCALP       18
FOOT        16
Name: count, dtype: int64
```

[37]:
```
df["location"].value_counts().sum()
```

[37]:
```
np.int64(2298)
```

[38]:
```
df["location"].isna().sum()
```

[38]:
```
np.int64(0)
```

[39]:
```
print(pd.crosstab(df["location"], df["label"]))
```

label	AKIEC	BCC	MEL	NEV	SCC	SEK
location						
ABDOMEN	3	2	2	19	2	8
ARM	91	44	7	10	21	19
BACK	16	91	14	86	17	24
CHEST	60	135	5	29	20	31
EAR	11	36	2	3	13	8
FACE	139	260	5	38	29	99
FOOT	4	0	0	6	6	0
FOREARM	261	67	5	10	28	21
HAND	78	19	2	6	19	2
LIP	4	10	0	3	6	0
NECK	18	48	3	8	10	6
NOSE	23	123	0	0	9	3
SCALP	4	0	0	9	1	4
THIGH	18	10	7	17	11	10

1 Split

[40]:
```
len(df["patient_id"])
```

[40]: 2298

[41]:
```
patient_ids = df["patient_id"].dropna().unique().to_numpy()
```

[42]:
```
rng.shuffle(patient_ids)
```

[43]:
```
df["patient_id"].isna().sum()
```

[43]: np.int64(0)

[44]:
```
n_patients = len(patient_ids)
n_train = int(TRAIN_RATIO * n_patients)
n_val = int(VAL_RATIO * n_patients)
```

[45]:
```
train_patients = patient_ids[:n_train]
val_patients = patient_ids[n_train:n_train + n_val]
test_patients = patient_ids[n_train + n_val:]
```

[46]:
```
train_ids = set(train_patients)
val_ids   = set(val_patients)
test_ids  = set(test_patients)

print("Train  Val :", train_ids & val_ids)
print("Train  Test:", train_ids & test_ids)
print("Val  Test  :", val_ids & test_ids)
```

```
Train  Val : set()
Train  Test: set()
Val  Test  : set()
```

[47]:
```
df["split"] = "unassigned"
df.loc[df["patient_id"].isin(train_patients), "split"] = "train"
df.loc[df["patient_id"].isin(val_patients), "split"] = "val"
df.loc[df["patient_id"].isin(test_patients), "split"] = "test"
```

[48]:
```
assert (df["split"] != "unassigned").all()
```

[49]:
```
print("Patients per split:")
print(df.groupby("split")["patient_id"].nunique())
```

```
Patients per split:
split
test     275
train    961
val      137
Name: patient_id, dtype: int64
```

[50]:
```
print("\nImages per split:")
print(df["split"].value_counts())
```

```
Images per split:
split
train    1620
test      457
val       221
Name: count, dtype: int64
```

[51]:
```
print("\nClass distribution per split:")
print(pd.crosstab(df["split"], df["label"]))
```

```
Class distribution per split:
label  AKIEC  BCC  MEL  NEV  SCC  SEK
split
test     139  162   18   46   43   49
train    520  611   30  166  133  160
val       71   72    4   32   16   26
```

[52]:
```
print(df.groupby(["split", "label"]).size().unstack(fill_value=0))
```

```
label  AKIEC  BCC  MEL  NEV  SCC  SEK
split
test     139  162   18   46   43   49
train    520  611   30  166  133  160
val       71   72    4   32   16   26
```

[53]:
```
pd.crosstab(df["age"], df["split"])
```

[53]:
```
split  test  train  val
age
6         0      2    0
8         0      2    0
9         0      3    0
10        4      1    0
12        0      2    0
..      …    …  …
89        0      7    0
90        0     12    2
91        0      7    0
92        0      1    3
94        0      3    0

[84 rows x 3 columns]
```

[54]:
```
pd.crosstab(df["sex"], df["split"])
```

[54]:
```
split    test  train  val
sex
FEMALE    159    536   58
MALE      142    524   75
unknown   156    560   88
```

[55]:
```
pd.crosstab(df["location"], df["split"])
```

[55]:
```
split     test  train  val
location
ABDOMEN      8     23    5
ARM         51    126   15
BACK        53    170   25
CHEST       41    205   34
EAR         14     55    4
FACE       126    398   46
FOOT         0     10    6
FOREARM     78    275   39
HAND        24     89   13
LIP          6     10    7
NECK        11     80    2
NOSE        22    121   15
SCALP        3     12    3
THIGH       20     46    7
```

[56]:
```
df.groupby(["split", "sex", "label"]).size()
```

[56]:

split	sex	label	
test	FEMALE	AKIEC	39
		BCC	79
		MEL	10
		NEV	8
		SCC	20
		SEK	3
	MALE	AKIEC	21
		BCC	83
		MEL	8
		NEV	1
		SCC	23
		SEK	6
	unknown	AKIEC	79
		NEV	37
		SEK	40
train	FEMALE	AKIEC	105
		BCC	295
		MEL	18
		NEV	32
		SCC	62
		SEK	24
	MALE	AKIEC	92
		BCC	316
		MEL	12
		NEV	24
		SCC	71
		SEK	9
	unknown	AKIEC	323
		NEV	110
		SEK	127
val	FEMALE	AKIEC	9
		BCC	32
		NEV	9
		SCC	8
	MALE	AKIEC	17
		BCC	40
		MEL	4
		NEV	1
		SCC	8
		SEK	5
	unknown	AKIEC	45
		NEV	22
		SEK	21

dtype: int64

[57]:
```
assert df["image_name"].is_unique
assert df["split"].isin(["train", "val", "test"]).all()
assert df["sex"].isin(["MALE", "FEMALE", "unknown"]).all()
assert df["label"].notna().all()
```

[58]:
```
leakage = (
    df.groupby("patient_id")["split"]
    .nunique()
    .gt(1)
    .sum()
)
print(f"Patients appearing in multiple splits: {leakage}")
```

```
Patients appearing in multiple splits: 0
```

[59]:
```
df=df[["dataset", "split", "image_name", "label", "binary_label", "patient_id",
       "age", "sex", "location"]]
```

[60]:
```
expected_cols = [
    "dataset",
    "split",
    "image_name",
    "label",
    "binary_label",
    "patient_id",
    "age",
    "sex",
    "location",
]
assert list(df.columns) == expected_cols
```

[61]:
```
print("=" * 60)
print("PADUFES20 metadata preprocessing completed")
print(f"Images   : {len(df)}")
print(f"Patients : {df['patient_id'].nunique()}")
print(f"Classes  : {df['label'].nunique()}")
print(df["split"].value_counts())
print("=" * 60)
```

```
============================================================
PADUFES20 metadata preprocessing completed
Images   : 2298
Patients : 1373
Classes  : 6
split
train    1620
test      457
val       221
```

```
Name: count, dtype: int64
============================================================
```

[62]:
```
print(df.isnull().sum())
```

```
dataset         0
split           0
image_name      0
label           0
binary_label    0
patient_id      0
age             0
sex             0
location        0
dtype: int64
```

[63]:
```
assert df["image_name"].is_unique
assert df["patient_id"].notna().all()
assert df["label"].notna().all()
assert df["binary_label"].isin([0, 1]).all()
assert df["split"].isin(["train", "val", "test"]).all()
```

[64]:
```
assert df.isnull().sum().sum() == 0
```

[65]:
```
df.to_csv("padufes20_metadata_clean.csv", index=False, encoding="utf-8")
```

[66]:
```
df
```

[66]:

	dataset	split	image_name	label	binary_label	patient_id	age	sex	location
0	padufes20	train	PAT_1516_1765_530.png	NEV	0	PAT_1516	8	unknown	ARM
1	padufes20	train	PAT_46_881_939.png	BCC	1	PAT_46	55	FEMALE	NECK
2	padufes20	val	PAT_1545_1867_547.png	AKIEC	1	PAT_1545	77	unknown	FACE
3	padufes20	test	PAT_1989_4061_934.png	AKIEC	1	PAT_1989	75	unknown	HAND
4	padufes20	train	PAT_684_1302_588.png	BCC	1	PAT_684	79	MALE	FOREARM
…	…	…	…	…	…	…	…	…	…
2293	padufes20	train	PAT_1708_3156_175.png	AKIEC	1	PAT_1708			
2294	padufes20	train	PAT_46_880_140.png	BCC	1	PAT_46			
2295	padufes20	val	PAT_1343_1217_404.png	SEK	0	PAT_1343			
2296	padufes20	test	PAT_326_690_823.png	BCC	1	PAT_326			
2297	padufes20	train	PAT_1714_3189_989.png	SEK	0	PAT_1714			

```
2293   73  unknown     HAND
2294   55   FEMALE    CHEST
2295   74  unknown  FOREARM
2296   58   FEMALE     FACE
2297   41  unknown    THIGH

[2298 rows x 9 columns]
```